

JA EDUCORE – ACADEMIC SERVICES PLATFORM

Integrantes:

Orozco Botero Adrián Enrique

Vargas Echeverria Jaber

Tecnología en Desarrollo de Sistemas de Información y de Software

Fundación Universitaria Colombo Internacional – Unicolombo

Profesor: Tena Jonathan.

UNICOLOMBO

FUNDACIÓN UNIVERSITARIA COLOMBO INTERNACIONAL

CARTAGENA

— SISTEMA JA | GESTIÓN EDUCATIVA DIGITAL —

Cartagena de Indias, Colombia

05 de septiembre de 2026

DESCRIPCIÓN DEL PROBLEMA

Actualmente, la gestión de solicitudes académicas dentro de una institución educativa puede presentar diferentes dificultades cuando los procesos se realizan mediante correos electrónicos, formularios u otros medios que no están integrados entre sí. Los estudiantes necesitan realizar trámites como solicitudes de certificados de estudio, revisión de notas, cancelación de asignaturas, paz y salvo y consultas sobre el estado de sus solicitudes. Sin embargo, cuando estos procesos no cuentan con un sistema centralizado, se puede generar pérdida de información, dificultad para realizar seguimiento y tiempos de respuesta elevados.

A partir de esta situación surge la necesidad de desarrollar un Sistema de Gestión de Solicitudes Académicas que permita organizar y centralizar estos procesos. El sistema propuesto, denominado **JA EduCore – Academic Services Platform**, busca facilitar la interacción entre estudiantes y funcionarios mediante diferentes funcionalidades. Entre ellas se encuentran el registro y autenticación de usuarios, la creación y consulta de solicitudes académicas, la generación de certificados, el registro y cancelación de asignaturas, la consulta del historial de solicitudes y la gestión de estas por parte de los funcionarios. Estas funcionalidades se encuentran definidas mediante requerimientos funcionales y no funcionales que establecen las condiciones que debe cumplir la solución.

Además de solucionar las necesidades funcionales, el sistema debe garantizar aspectos relacionados con su calidad. Se plantea que la base de datos pueda manejar una cantidad considerable de información, que los certificados puedan generarse en formato PDF, que la plataforma sea responsive, que las operaciones tengan tiempos de respuesta adecuados y que la información de los usuarios cuente con medidas de seguridad. También se considera importante mantener una interfaz amigable y aplicar herramientas y técnicas que permitan construir una solución más completa.

Por esta razón, el problema no consiste únicamente en crear una aplicación que permita registrar solicitudes, sino en diseñar una solución que sea organizada, mantenible y capaz de adaptarse a futuros cambios. Para esto se plantea el uso de Scrum como metodología de desarrollo, MVC como modelo de diseño y una arquitectura modular basada en componentes y responsabilidades separadas. De esta manera, el sistema puede dividirse en áreas como autenticación, usuarios, solicitudes académicas, historial, gestión, notificaciones, reportes y persistencia de datos, permitiendo que cada componente tenga una responsabilidad definida. Finalmente, la implementación de pruebas permite comprobar si las funcionalidades desarrolladas realmente responden a los requerimientos establecidos.

A través de casos de prueba positivos y negativos se pueden identificar tanto los comportamientos correctos como aquellos aspectos que requieren mejoras. En el análisis realizado se identificó, por ejemplo, que debía fortalecerse la validación relacionada con el filtrado de solicitudes por parte de los funcionarios y que era necesario mejorar la organización de la numeración de algunos requerimientos y pruebas.

MATRIZ DE REQUERIMIENTOS

ID	REQUERIMIENTO	TIPO	PRIORIDAD	CRITERIO DE ACEPTACIÓN
RF-01	El sistema debe permitir registrar estudiantes.	Funcional	Alta	El estudiante no debe aparecer registrado en el sistema.
RF-02	El sistema debe permitir que el estudiante inicie sesión.	Funcional	Alta	El estudiante debe tener una cuenta, y el sistema debe validar las credenciales.
RF-03	El sistema debe permitir al usuario recuperar su contraseña.	Funcional	Media	El sistema debe verificar que la cuenta del estudiante esté creada.
RF-04	El sistema debe permitir al estudiante crear solicitudes académicas de revisión de notas.	Funcional	Alta	El estudiante debe tener sesión activa y debe tener al menos una nota del/los periodo/s transcurrido.
RF-05	El sistema debe permitir al estudiante generar certificados de estudios.	Funcional	Alta	El estudiante debe tener la sesión activa y validar que tenga matrícula vigente.
RF-06	El sistema debe permitir al estudiante registrar sus asignaturas.	Funcional	Alta	El estudiante debe tener la sesión activa y validar que tenga matrícula financiera vigente.
RF-07	El sistema debe permitir al estudiante cancelar sus asignaturas	Funcional	Alta	El estudiante debe tener la sesión activa y que las asignaturas a cancelar estén previamente asignadas.
RF-08	El sistema debe permitir al estudiante realizar solicitudes de paz y salvo.	Funcional	Alta	El estudiante debe tener sesión activa y tener matrícula financiera vigente.
RF-09	El sistema debe permitir al estudiante descargar los certificados generados.	Funcional	Alta	El estudiante debe tener sesión activa y que el documento ya se halla generado.
RF-10	El sistema debe permitir que el	Funcional	Alta	El estudiante debe tener sesión activa, debe existir la solicitud que quiere cancelar y

	estudiante cancele solicitudes.			la solicitud no debe estar ya respondida.
RF-11	El sistema debe permitir que el estudiante actualice su información personal.	Funcional	Media	El estudiante debe tener sesión activa y el sistema debe verificar la información nueva que se modificará la anterior.
RF-12	El sistema debe permitir que el estudiante acepte términos y condiciones, cookies y derechos del sistema.	Funcional	Alta	El sistema debe mostrar los términos y condiciones, cookies y derechos del sistema con un checkbox.
RF-13	El sistema debe permitir que el usuario consulte una solicitud.	Funcional	Alta	El estudiante debe tener sesión activa y debe recibirse el ID de la solicitud.
RF-14	El sistema debe permitir al estudiante tener un dashboard principal con un resumen de sus movimientos en la cuenta.	Funcional	Media	El estudiante debe tener sesión activa, y debe tener al menos un movimiento de cada tipo para que se pueda registrar en el dashboard y no aparezca nulo – vacío.
RF-15	El sistema debe permitir registrar estudiantes con sus datos personales y académicos	Funcional	Alta	El sistema debe permitir diferenciar el acceso según el tipo de usuario, estudiante o funcionario.
RF-16	El sistema debe permitir a los usuarios iniciar sesión mediante usuario y contraseña.	Funcional	Alta	El sistema valida las credenciales y permite el acceso cuando son correctas.
RF-17	El sistema debe permitir diferenciar el acceso según el tipo de usuario, estudiante o funcionario.	Funcional	Alta	Cada usuario visualiza únicamente las opciones correspondientes a su rol.
RF-18	El sistema debe permitir a los estudiantes crear solicitudes académicas.	Funcional	Alta	La solicitud se registra y recibe un número identificador único.

RF-19	El sistema debe permitir seleccionar el tipo de solicitud académica al momento de crearla.	Funcional	Alta	El estudiante puede seleccionar entre los tipos de solicitudes disponibles, como certificado, revisión de notas, cancelación de asignatura o paz y salvo.
RF-20	El sistema debe permitir mantener un historial de los cambios realizados sobre una solicitud.	Funcional	Media	El sistema conserva los estados anteriores y permite consultar el historial de la solicitud.
RF-21	El sistema debe permitir consultar una solicitud mediante su número identificador.	Funcional	Media	Al ingresar un número válido, el sistema muestra la información correspondiente a la solicitud.
RF-22	El sistema debe permitir a los funcionarios filtrar solicitud por estado	Funcional	Media	El funcionario puede visualizar únicamente las solicitudes que correspondan al estado seleccionado.
RNF-01	La base de datos del sistema debe estar preparada para manejar bastante información.	No funcional	Alta	La base de datos debe estar creada con buenas prácticas de diseño e índices.
RNF-02	Los certificados deben generarse en formato PDF, diseñada la plantilla por los creadores del proyecto.	No funcional	Media	Se debe usar alguna librería o servicio que facilite la generación del PDF, y el PDF ya debe estar diseñado su diseño.
RNF-03	El sistema debe ser totalmente responsive.	No funcional	Alta	Usar JQuerys, grid o flexbox en el diseño de la web.
RNF-04	El sistema debe responder ante alguna solicitud u operación en un tiempo de 3 segundos.	No funcional	Alta	La base de datos del sistema debe tener las operaciones correctas y escritas de la mejor manera, usando buenas prácticas.
RNF-05	La interfaz del sistema debe ser amigable y estéticamente bonita.	No funcional	Media	Respetando los colores del branding establecidos por los creadores del proyecto.

RNF-06	Las contraseñas guardadas en la base de datos deben guardarse en formato hash.	No funcional	Baja	Implementar alguna API que facilite este proceso.
RNF-07	El sistema debe tener implementada herramientas un poco más avanzadas.	No funcional	Media	Usar frameworks, librerías, APIS, Saas u otras herramientas / técnicas avanzadas.



UNICOLOMBO

FUNDACIÓN UNIVERSITARIA COLOMBO INTERNACIONAL

CARTAGENA

— SISTEMA JA | GESTIÓN EDUCATIVA DIGITAL —

CASOS DE PRUEBA

ID	REQUERIMIENTO	ENTRADA	RESULTADO OBTENIDO	ESTADO
CP-01	RF-01	Los datos de registros están digitados y fueron validados	El sistema registra al estudiante correctamente con todos los datos proporcionado en el proceso de creación de cuenta.	
CP-02	RF-01	Ya existe un correo en el sistema.	El sistema rechaza el registro y notifica que el correo está en uso.	
CP-03	RF-01	El registro está con campos obligatorios vacíos.	El sistema rechaza el registro y notifica que todos los campos que son obligatorios deben contener información.	
CP-04	RF-02	El usuario y contraseña son válidos.	El sistema permite al estudiante el ingreso por sus credenciales.	
CP-05	RF-02	El usuario es válido pero la contraseña es incorrecta.	Rechaza el acceso y notifica que la contraseña proporcionada es incorrecta.	
CP-06	RF-02	El usuario no sale registrado en el sistema.	Rechaza el acceso al sistema y notifica que el usuario no existe en la aplicación.	
CP-07	RF-03	Correo de una cuenta existente.	El sistema inicia el proceso de recuperación de contraseña.	
CP-08	RF-03	Correo de una cuenta no registrada.	El sistema notifica el error diciendo que la cuenta no existe.	
CP-09	RF-04	Sesión activa y al menos una nota del periodo transcurrido.	El sistema crea la solicitud del proceso de revisión de notas que tiene el estudiante, durante el periodo.	
CP-10	RF-04	El estudiante no tiene notas registradas en el periodo.	El sistema rechaza la creación de la solicitud y notifica que no hay notas aún.	
CP-11	RF-05	Sesión y matrícula vigente	El sistema realiza la generación de certificado de estudio en PDF.	

CP-12	RF-05	Sesión activa con matrícula no vigente.	El sistema rechaza la solicitud de generación del certificado.	
CP-13	RF-06	Sesión activa y matrícula financiera vigente.	El estudiante registra las asignaturas que él mismo decidió seleccionar en el sistema.	
CP-14	RF-06	Sesión activa y no tiene matrícula financiera vigente.	El sistema rechaza el proceso de registro de asignaturas que el estudiante quería seleccionar.	
CP-15	RF-07	Asignatura que ya tenía asignada el estudiante.	Cancela la asignación al estudiante y le notifica que ya la tiene asociada a su horario.	
CP-16	RF-07	Intento de cancela una asignatura no asignada.	El sistema rechaza la cancelación de la asignatura y muestra un mensaje sobre el error.	
CP-17	RF-08	Sesión activa y el estudiante tiene matrícula financiera vigente.	El sistema genera la solicitud de paz y salvo.	
CP-18	RF-08	Sesión activa pero el estudiante no tiene matrícula financiera vigente.	El sistema la solicitud de paz y salvo y notifica el error.	
CP-19	RF-09	Sesión activa y certificado que ya fue generado.	Permite la descarga del certificado.	
CP-20	RF-09	Sesión activa con certificado aún no generado.	El sistema rechaza la descarga y notifica el error.	
CP-21	RF-10	Sesión activa y solicitud existente sin responder.	Cancela la solicitud correctamente.	
CP-22	RF-10	Intento de cancelar una solicitud ya respondida.	El sistema rechaza la cancelación.	
		Intento de cancelar una solicitud con un		

CP-23	RF-10	identificador inexistente.	El sistema muestra un error de solicitud no encontrada.	
CP-24	RF-11	Sesión activa y datos nuevos que son válidos.	Actualiza la información personal del estudiante.	
CP-25	RF-11	Sesión activa con datos nuevos en formatos incorrectos.	El sistema rechaza la actualización y muestra error de validación.	
CP-26	RF-12	El estudiante marca el checkbox de términos, condiciones y cookies.	El sistema permite continuar con el registro e ingreso.	
CP-27	RF-12	Estudiante intenta continuar sin marca el checkbox.	El sistema bloque el avance y exige aceptar los términos	
CP-28	RF-13	Sesión activa y el identificador de solicitud es válido.	Muestra la información de la solicitud consultada.	
CP-29	RF-13	Sesión activa con ID de solicitud inexistente.	El sistema muestra un error de solicitud no encontrada.	
CP-30	RF-14	Sesión activa con al menos un movimiento de cada tipo.	El dashboard muestra el resumen completo de movimientos.	
CP-31	RF-14	Sesión activa sin movimientos registrados.	El dashboard se muestra vacío o nulo en lugar de tener datos.	
CP-32	RF-15	El estudiante no tiene datos académicos relacionados con la institución	El sistema rechaza el registro al no encontrar datos relacionados con la institución	
CP-33	RF-15	El estudiante cuenta sus datos respectivos datos académicos par un registro exitoso	El sistema realiza el registro al estudiante de manera exitosa	

CP-34	RF-18	El estudiante completa la información válida requerida para una nueva solicitud académica y presiona "Crear".	El sistema registra la solicitud exitosamente y genera un número identificador único asociado a la gestión	
CP-35	RF-18	El usuario ingresa usuario o contraseña incorrectos	El sistema arroja un mensaje de alerta diciendo que el usuario y/o contraseña son incorrectas	
CP-36	RF-18	El estudiante intenta enviar el formulario de solicitud dejando campos obligatorios vacíos.	El sistema rechaza la solicitud, muestra mensajes de validación y no asigna identificador único.	
CP-37	RF-19	El estudiante despliega la lista de tipos de solicitud al crear una nueva gestión.	El sistema muestra las opciones válidas (Certificado, Revisión de notas, Cancelación de asignatura, Paz y salvo).	
CP-38	RF-19	El estudiante selecciona el tipo "Certificado" y confirma el registro de la solicitud.	El sistema guarda la solicitud asociándola correctamente a la categoría seleccionada.	
CP-39	RF-20	El usuario solicita consultar el historial de cambios de una solicitud que ha pasado por diferentes estados.	El sistema despliega la lista de estados anteriores con sus respectivas fechas y modificaciones conservadas.	
CP-40	RF-20	El usuario intenta consultar el historial de una solicitud recién creada	El sistema muestra únicamente el registro inicial de creación en el historial sin errores.	

		que no ha sufrido modificaciones.		
CP-41	RF-21	El usuario ingresa un número identificador único válido en el campo de búsqueda de solicitudes.	El sistema ubica la solicitud y despliega en pantalla la información completa correspondiente.	
CP-42	RF-21	El sistema ubica la solicitud y despliega en pantalla la información completa correspondiente.	El sistema muestra un mensaje indicando que no se encontró ninguna solicitud asociada al identificador ingresado.	



UNICOLOMBO

FUNDACIÓN UNIVERSITARIA COLOMBO INTERNACIONAL

CARTAGENA

— SISTEMA JA | GESTIÓN EDUCATIVA DIGITAL —

RESULTADOS DE LAS PRUEBAS

¿QUE REQUERIMIENTOS FUERON CUMPLIDOS?

RF-01: El sistema debe permitir registrar estudiantes con sus datos personales y académicos.

RF-02: El sistema debe permitir a los usuarios iniciar sesión mediante usuario y contraseña.

RF-03: El sistema debe permitir diferenciar el acceso según el tipo de usuario, estudiante o funcionario.

RF-04 / RF-18: El sistema debe permitir a los estudiantes crear solicitudes académicas.

RF-05 / RF-19: El sistema debe permitir seleccionar el tipo de solicitud académica al momento de crearla.

RF-06 / RF-20: El sistema debe permitir mantener un historial de los cambios realizados sobre una solicitud.

RF-07 / RF-21: El sistema debe permitir consultar una solicitud mediante su número identificador.

RF-15, RF-16 y RF-17: Cobertura de registro de datos institucionales, inicios de sesión con manejo de errores/validación de vacíos y autenticación basada en roles

¿QUÉ REQUERIMIENTOS NO FUERON CUMPLIDOS?

RF-08: "El sistema debe permitir a los funcionarios filtrar solicitud por estado". En la matriz documentada no se especificó un caso de prueba para la funcionalidad de filtrado de solicitudes por parte de funcionarios.

Requerimientos No Funcionales (RNF): No se documentaron casos de prueba para validar criterios de rendimiento, seguridad de la infraestructura o tiempos de respuesta.

¿QUE PRUEBAS FUERON POSITIVAS?

pruebas orientadas a verificar el funcionamiento correcto ante entradas válidas ("camino feliz"):

- **CP-01:** Registro exitoso de estudiante con datos digitados y validados.
- **CP-04:** Inicio de sesión exitoso con usuario y contraseña válidos.
- **CP-07:** Inicio del proceso de recuperación de contraseña con correo existente.
- **CP-09:** Creación de solicitud de revisión de notas con sesión activa y notas registradas.
- **CP-11:** Generación de certificado de estudio en PDF con matrícula vigente.
- **CP-13:** Registro de asignaturas seleccionadas con matrícula financiera vigente.

- **CP-17:** Registro exitoso del estudiante al contar con datos académicos institucionales.
- **CP-21:** Bienvenida y acceso al sistema con credenciales correctas.
- **CP-23:** Inicio de sesión exitoso al ingresar credenciales y rol respectivo correcto.
- **CP-24:** Registro exitoso de solicitud académica y asignación de identificador único al presionar "Crear" con información válida.
- **CP-26:** Despliegue correcto de los tipos de solicitud (Certificado, Revisión de notas, Cancelación de asignatura, Paz y salvo).
- **CP-27:** Guardado de solicitud asociada correctamente al tipo seleccionado.
- **CP-28:** Despliegue del historial de cambios con estados anteriores, fechas y modificaciones.
- **CP-29:** Muestra del registro inicial de creación en el historial para solicitudes sin modificaciones.
- **CP-30:** Ubicación y despliegue de información al ingresar un número identificador único válido.

¿QUE PRUEBAS FUERON NEGATIVAS?

- CP-02:** Rechazo de registro por intento de usar un correo ya registrado.
- CP-03:** Rechazo de registro al dejar campos obligatorios vacíos.
- CP-05:** Rechazo de acceso por contraseña incorrecta.
- CP-06:** Rechazo de acceso por usuario no registrado.
- CP-08:** Notificación de error en recuperación de contraseña para cuentas no registradas.
- CP-10:** Rechazo de creación de solicitud por ausencia de notas registradas.
- CP-12:** Rechazo de certificado por matrícula no vigente.
- CP-14:** Rechazo de registro de asignaturas por falta de matrícula financiera vigente.
- CP-15:** Cancelación de asignación al detectar asignatura previamente asociada.
- CP-16:** Rechazo de registro por falta de datos académicos en la institución.
- CP-18:** Alerta de campos obligatorios al dejar en blanco el usuario o la contraseña.
- CP-19:** Alerta por datos de usuario o contraseña incorrectos.
- CP-20:** Alerta por intento de inicio de sesión de usuario no registrado.
- CP-22:** Rechazo de acceso cuando la cuenta no corresponde al rol seleccionado.
- CP-25:** Rechazo de solicitud al enviar el formulario con campos obligatorios vacíos.

CP-31: Notificación de "no encontrado" al ingresar un número identificador inexistente o inválido

¿QUE CAMBIOS DEBERIA REALIZARSE EL SOFTWARE?

Normalización de la numeración de requerimientos: Corregir el desfase en los identificadores entre el documento de Requisitos y los Casos de Pruebas (por ejemplo, alinear el RF-04/RF-18, RF-05/RF-19, RF-06/RF-20 y RF-07/RF-21).

Desarrollo del módulo de filtrado: Diseñar e incorporar las pruebas y componentes necesarios para cumplir con la funcionalidad de filtrado de solicitudes por estado para los funcionarios (RF-08).

Ajuste de validaciones en casos de prueba: Corregir duplicidades observadas en la redacción de entradas/resultados en casos de prueba como el CP-31.

¿CÓMO DEMOSTRARÍAN ANTE LA ORGANIZACIÓN QUE LA SOLUCIÓN CUMPLE LOS REQUERIMIENTOS?

Matriz de Trazabilidad: Presentando el cruce entre los Requisitos Funcionales (RF-01 al RF-07/RF-15 a RF-21) y los Casos de Prueba mapeados (CP-01 al CP-31).

Protocolo de Aceptación (UAT): Ejecutando cada caso de prueba frente a los interesados para verificar que las salidas obtenidas por el sistema correspondan con los resultados esperados descritos en la documentación.

UNICOLOMBO

FUNDACIÓN UNIVERSITARIA COLOMBO INTERNACIONAL

CARTAGENA

— SISTEMA JA | GESTIÓN EDUCATIVA DIGITAL —

METODOLOGÍA SELECCIONADA

Se optó por la metodología **SCRUM**.

¿Por qué?

Siendo muy cercano al contexto de este sistema, el tipo de trabajo que tiene esta metodología va muy ligado a la forma de trabajo. Ya que esta metodología consiste en entregar valor o funcionalidades de forma rápida e incremental mediante ciclos cortos de trabajos llamada sprint.

Donde sus 3 principios son:

- Trabajo iterativo e incremental.
- Colaboración continua.
- Transparencia e inspección.

En este proyecto se trabaja de forma rápida porque el valor debe entregarse a más tardar 1 semana después de la asignación, tiene bastantes actividades que deben distribuirse, es en equipo y debe verificarse entre los mismos integrantes.

Evidencia de roles aplicados al trabajo

- **Product Owner (Propietario del producto):** encargado de asignar tareas y qué se debe construir / hacer; priorizar tareas según su necesidad y gestionar el producto backlog.
Encargado: Orozco Botero Adrián Enrique.
- **Scrum Master:** facilita la aplicación de Scrum; elimina obstáculos que afecten al equipo y promueve las buenas prácticas ágiles.
Encargado: Orozco Botero Adrián Enrique.
- **Equipo de desarrollo:** grupo multidisciplinario que realiza el trabajo necesario para crear el producto y se autoorganiza para alcanzar los objetivos principales del sprint.
Grupo de desarrollo: Vargas Echeverria Jaber y Orozco Botero Adrián Enrique.

Eventos más aplicados en este proyecto: **sprint planning (planeación antes de hacer)**, **sprint (hacer)**, **sprint review (hacer)** y **sprint retrospective (verificar)**.

MODELO DE DISEÑO

Para el modelo de diseño se ha optado por MVC (Modelo – Vista – Controlador).

Ya que ayuda a mantener la modularidad y mantenibilidad del proyecto. Siendo una ventaja también su escalabilidad para la programación orientada a objetos.

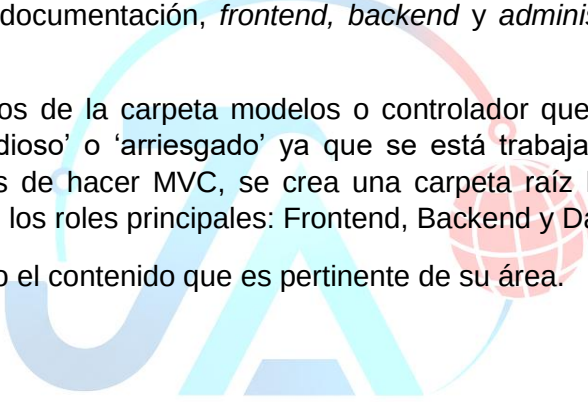
Permitiendo separar la lógica del negocio, gestión de datos y las interfaces del usuario. Siendo el modelo quién represente la tabla de bases de datos y datos del sistema; la vista responde por la interacción con el usuario y los controladores gestionan las solicitudes y la comunicación entre las diferentes partes de la aplicación.

Pero, a nivel personal, se ha optado por una variante que es: separar responsabilidades + MVC.

Ya que en un grupo de trabajo siempre existen roles, y estos normalmente se segmentan en grupos como: tester, documentación, *frontend*, *backend* y *administrador de base de datos*.

Al tener MVC hay archivos de la carpeta modelos o controlador que debe manipular el frontend, siendo algo 'tedioso' o 'arriesgado' ya que se está trabajando en un entorno diferente. Por ende antes de hacer MVC, se crea una carpeta raíz llamada Src, con 3 subcarpetas que contiene los roles principales: Frontend, Backend y Database.

Cada subcarpeta teniendo el contenido que es pertinente de su área.



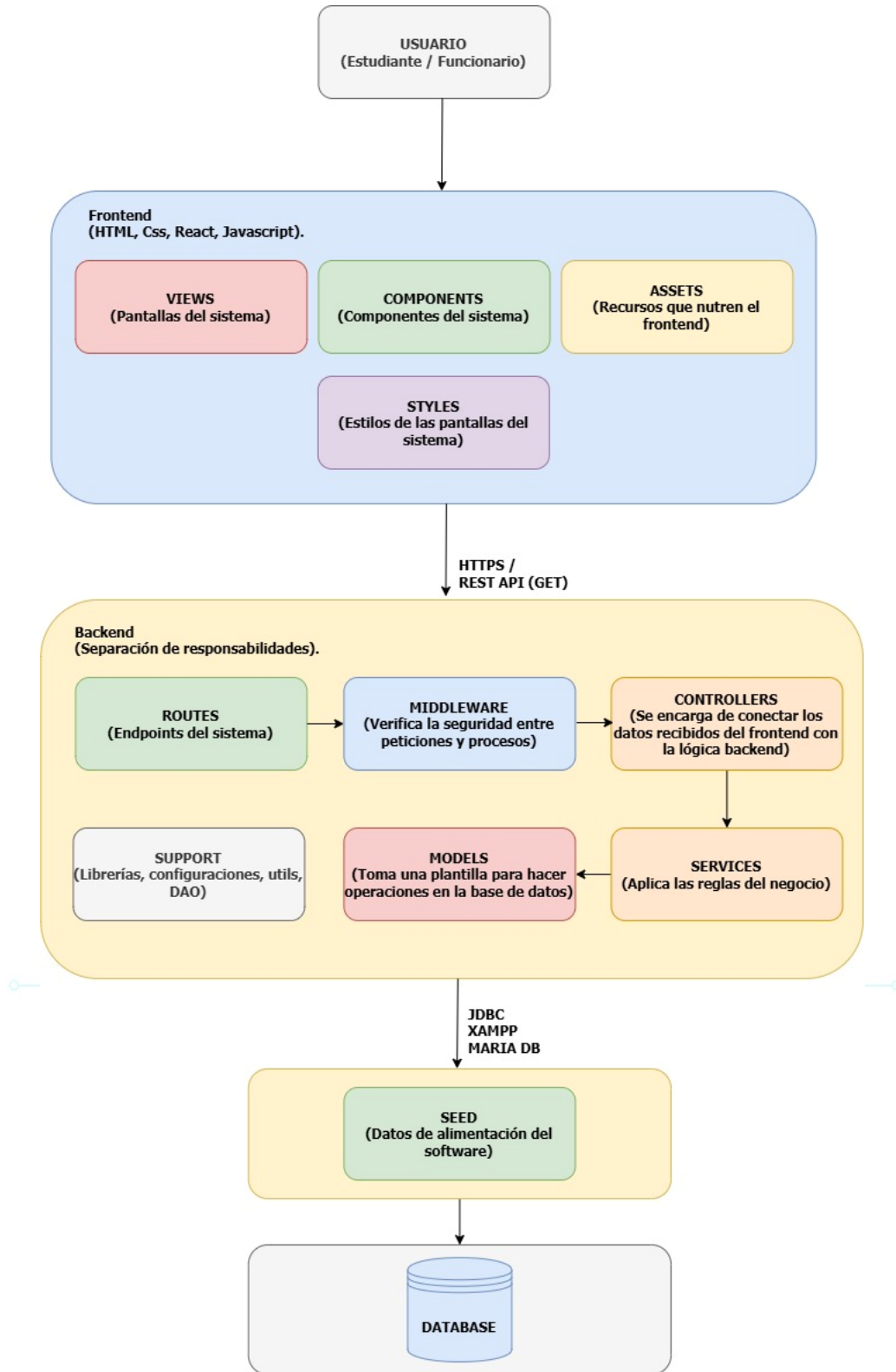
UNICOLOMBO

FUNDACIÓN UNIVERSITARIA COLOMBO INTERNACIONAL

CARTAGENA

— SISTEMA JA | GESTIÓN EDUCATIVA DIGITAL —

DIAGRAMA MVC Y DE ARQUITECTURA



DESCRIPCIÓN DE COMPONENTES

Componente	Responsabilidad	Entradas (Inputs)	Salidas (Outputs)	Dependencias	Interfaz Definida
Autenticación y Seguridad	Gestionar el acceso, validación de credenciales y control de acceso basado en roles (estudiante/funcionario).	Credenciales (usuario, contraseña, rol seleccionado).	Token JWT / Sesión activa, Mensaje de error/bienvenida.	Persistencia de Datos.	IAuthService (login(), validateToken())
Gestión de Usuarios	Registrar y administrar la información personal y académica de estudiantes y funcionarios.	Formulario de datos personales y académicos.	Confirmación de registro, Alertas de validación de correo.	Persistencia de Datos.	IUserService (registerUser(), getUserProfile())
Solicitudes Académicas	Permitir la creación, selección de tipos y consulta individual de solicitudes académicas por identificador único.	Tipo de solicitud, datos requeridos, ID único de búsqueda.	ID único generado, Detalle de la solicitud, Alertas de campos obligatorios.	Gestión de Usuarios, Notificaciones, Persistencia de Datos.	IRequestService (createRequest(), getRequestById())

Componente	Responsabilidad	Entradas (Inputs)	Salidas (Outputs)	Dependencias	Interfaz Definida
Historial y Auditoría	Conservar y listar el registro cronológico de estados y modificaciones realizadas sobre una solicitud.	ID de la solicitud, evento o cambio de estado realizado.	Trazabilidad histórica (lista de estados anteriores con fechas y notas).	Solicitudes Académicas, Persistencia de Datos.	IHistoryService (logStateChange(), getHistoryByRequestId())
Gestión y Filtrado	Permitir a los funcionarios evaluar, actualizar estados y filtrar solicitudes según su estado actual.	Parámetros de filtro (estado, fecha), nuevo estado asignado.	Lista filtrada de solicitudes, Confirmación de actualización.	Solicitudes Académicas, Notificaciones, Persistencia de Datos.	IGestionService (filterRequestsByStatus(), updateStatus())
Notificaciones	Informar de manera automática a los usuarios sobre cambios de estado o confirmaciones del proceso.	Eventos del sistema (ej. "Solicitud Creada", "Estado Cambiado"), ID del destinatario.	Mensajes informativos en plataforma / Envíos de correo electrónico.	Ninguna (Escucha eventos del sistema).	INotificationService (sendNotification())

Componente	Responsabilidad	Entradas (Inputs)	Salidas (Outputs)	Dependencias	Interfaz Definida
Reportes y Estadísticas	Generar métricas, consolidados e indicadores estadísticos sobre las solicitudes para gestión institucional.	Filtros de fecha, tipo de solicitud o estado de gestión.	Consolidado de datos e indicadores estandarizados.	Persistencia de Datos.	IReportService (generateReport())
Persistencia de Datos	Abstraer las transacciones y consultas con la base de datos relacional (ORM / DAO).	Consultas, sentencias, entidades de datos.	Registros procesados, Respuestas directas de la base de datos.	Base de Datos (MySQL / SQL Server).	IRepository<T> (find(), save(), update())

2. PREGUNTA DE ANÁLISIS

¿QUÉ PASARÍA SI MODIFICAMOS EL MÓDULO DE NOTIFICACIONES?
¿TENDRÍAMOS QUE MODIFICAR TODO EL SISTEMA?

Respuesta:

No, no se tendría que modificar todo el sistema. Esta independencia se logra gracias a que la arquitectura está diseñada bajo los principios de bajo acoplamiento y alta cohesión:

- **Bajo Acoplamiento:** Los demás componentes del sistema (como *Solicitudes Académicas* o *Gestión y Filtrado*) no conocen la implementación interna del módulo de *Notificaciones*; únicamente interactúan con él a través de una interfaz genérica (INotificationService) o mediante un esquema de arquitectura orientada a eventos. Si en el futuro se decide cambiar la forma de enviar notificaciones (por ejemplo, pasar de correo electrónico a SMS o WhatsApp), solo se modificará el código interno del componente de Notificaciones sin afectar los controladores, modelos o vistas del resto

del proyecto.

- **Alta Cohesión:** El módulo de *Notificaciones* tiene una responsabilidad única y claramente delimitada: gestionar e informar eventos al usuario. No mezcla lógica de negocio ni manipula datos de registros académicos o autenticación, lo que garantiza que cualquier cambio permanezca aislado, facilitando el mantenimiento y evitando errores en cascada.



ANÁLISIS DE MODULARIDAD

Ante el requerimiento institucional de incorporar un Módulo de PQRS y un Sistema de Notificaciones por Correo Electrónico en la segunda fase, se presenta el análisis de impacto y adaptabilidad de la arquitectura propuesta:

1. ¿Qué componentes nuevos se necesitan?

- **Módulo de PQRS (Peticiónes, Quejas, Reclamos y Sugerencias):** Encargado de la lógica de negocio exclusiva para el registro, categorización, asignación a funcionarios y seguimiento de casos de PQRS.
- **Proveedor / Adaptador de Correo Electrónico (Email Service Provider / SMTP Adapter):** Un componente especializado dentro o conectado al módulo de Notificaciones para gestionar el formateo de plantillas HTML y la conexión con servidores de correo (como SMTP, SendGrid o PHPMailer).

2. ¿Qué componentes existentes deben modificarse?

- **Módulo de Notificaciones:** No requiere reescribir su lógica previa, pero se debe extender agregando el canal de envío por correo electrónico (implementando la interfaz de notificaciones para soportar este nuevo medio).
- **Controladores y Vistas de la Interfaz:** Agregar únicamente las nuevas pantallas (vistas) y rutas de control necesarias para que el usuario pueda interactuar con el formulario de PQRS.

3. ¿Qué componentes pueden reutilizarse?

- **Autenticación y Seguridad:** Reutilizable al 100%. Mantiene el control de acceso, inicio de sesión y validación de roles sin requerir modificaciones.
- **Gestión de Usuarios:** Reutilizable al 100%. Proporciona la información del estudiante o funcionario que radica la PQRS.
- **Historial y Auditoría:** Reutilizable al 100%. Sirve para registrar la trazabilidad de estados y respuestas de las PQRS exactamente igual que con las solicitudes académicas.
- **Persistencia de Datos:** Reutilizable al 100%. El repositorio genérico se usa para realizar las operaciones CRUD en las nuevas tablas de la base de datos sin cambiar su código base.
- **Reportes y Estadísticas:** Reutilizable en su estructura base para incluir la consolidación de métricas de PQRS.

4. ¿La arquitectura propuesta soporta fácilmente este cambio?

Sí, totalmente. Al estar basada en una arquitectura en capas con el patrón MVC y módulos desacoplados mediante interfaces, la inclusión de un nuevo módulo (PQRS) se realiza mediante un proceso de **extensión modular**. La estructura no se quiebra ni requiere reestructurar los componentes existentes.

5. ¿Se está cumpliendo el principio Open/Closed (Abierto/Cerrado)?

Sí, se cumple estrictamente. El principio Open/Closed establece que las entidades de software deben estar *abiertas a la extensión pero cerradas a la modificación*. Para agregar PQRS y notificaciones por correo, se extiende el sistema agregando nuevas clases y módulos que implementan las interfaces existentes (INotificationService, IRepository), sin necesidad de modificar ni alterar el código fuente funcional de los módulos ya desarrollados (Autenticación, Registro o Solicitudes Académicas).

6. ¿Existe demasiado acoplamiento?

No, no existe demasiado acoplamiento; el acoplamiento es bajo. Cada componente interactúa a través de interfaces y eventos. El módulo de PQRS o el de Solicitudes simplemente notifica que ocurrió un evento ("PQRS Radicada" o "Solicitud Creada") y el componente de Notificaciones lo procesa de manera independiente sin que los módulos principales conozcan los detalles técnicos del envío del correo.

7. ¿Cómo modificarían el diagrama de arquitectura?

- **En la Capa de Lógica de Negocio / Módulos:** Se añade un bloque paralelo a *Solicitudes Académicas* denominado **Módulo de PQRS**.
- **En el Módulo de Notificaciones:** Se añade una sub-capa o adaptador para el **Servicio de Correo (SMTP / Email Provider)**.
- **En la Capa de Presentación (Vistas/Controladores):** Se agrega el controlador PQRSController y sus respectivas vistas asociadas (CrearPQRS, ConsultarPQRS).
- **En la Base de Datos:** Se conectan las nuevas entidades relacionales (PQRS, TipoPQRS, RespuestaPQRS) a la capa de Persistencia existente.

PATRONES Y BUENAS PRÁCTICAS

1. MVC

En el sistema se puede aplicar el patrón MVV para separar las interfaces, lógica de control y los accesos a los datos.

- **View:** corresponde al frontend, especialmente a los views, donde se encuentran todas las pantallas que usarán los estudiantes y funcionarios, además de en esa carpeta estar acompañada por componentes, recursos, estilos, etc.
- **Model:** corresponde a la representación de los datos almacenados en una base de datos. Representando cada una de las tablas existentes en el sistema.
- **Controller:** correspondiente al backend, siendo el puente entre el frontend y la lógica de servicios del backend.
- **Services:** también dentro de la capa backend, ya que concentra la lógica del negocio, evitando que los controladores tengan muchas responsabilidades.

Aportando al bajo acoplamiento ya que se pueden hacer modificaciones en diferentes áreas sin que se afecte otro componente. Además de que se hará fácil la creación de funciones según el tipo de rol.

2. SOLID

2.1. Single Responsibility Principle (SRP): el principio de responsabilidad única establece que cada módulo o clase debe tener una responsabilidad específica. Reflejándose en la arquitectura propuesta porque cada componente tiene una función determinada.

- **Routes:** endpoints disponibles.
- **Middleware:** seguridad y procesamiento de peticiones.
- **Controllers:** recibe solicitudes y coordina respuestas.
- **Services:** lógica del negocio.
- **Models:** representación de los datos.
- **Views:** pantallas del sistema.

Y así con cada una de las carpetas del proyecto. Ejemplo: el controller no se debe encargar de hacer consultas a la base de datos ni realizar la aplicación de todas las reglas del negocio, eso lo hace models y services. Facilitando el mantenimiento del sistema porque existen diferentes funcionalidades.

2.2. Open/Closed Principle (OCP): este principio dice que los componentes están abiertos para extensión, pero cerrados para modificación se aplican en los services y controllers. Ya que un estudiante puede realizar diferentes solicitudes como: certificados, revisión de notas, paz y salvo, etc.

Pero el RF-05 da soporte de que un estudiante puede seleccionar entre diferentes tipos de solicitudes. Por ende, si se agrega un nuevo tipo de solicitud, se debe diseñar la lógica de que se pueda agregar ese nuevo tipo, sin modificar las que ya están existentes

2.3. Dependency Inversion Principle (DIP): hace que los componentes de alto nivel no tengan alto acoplamiento con las implementaciones concretas, sino de abstracciones.

En Ja Educare, se aplica de esta forma:

- El controller recibe la petición.
- El service aplica las reglas del negocio.
- El model se encarga de la interacción con los datos.
- La base de datos almacena la información.

Haciendo que el controller dependa de servicios necesarios en lugar de que contenga directamente la lógica o instrucciones sql.

2.4. Modularidad: la arquitectura propuesta aplica modularidad, ya que el sistema se divide en módulos con responsabilidades específicas.

- Frontend.
- Backend.
- Database.

Permitiendo trabajar y mantener cada parte del sistema de forma independiente.

¿CUÁL SERÍA EL PROBLEMA DE CONSTRUIR TODO EL SISTEMA DENTRO DE UNA SOLA CLASE O UN SOLO MÓDULO?

Existiría algo que se llama alto acoplamiento, y ese archivo manejaría gran cantidad de responsabilidades en un mismo lugar. Haciendo que el código sea difícil de mantener, probar, modificar y ampliar. Además de que un cambio realizado para solucionar o agregar una funcionalidad podría afectar otras partes del sistema.

En cambio, si se usan buenos patrones y prácticas como la modularidad, se crearían componentes con responsabilidad específica. Facilitando que nuevas funciones puedan incorporarse sin modificar todo el sistema

CONCLUSIÓN

El desarrollo de **JA EduCore** permitió comprender que construir un sistema de software no consiste solamente en programar las funcionalidades solicitadas, sino en realizar previamente un análisis que permita definir qué debe hacer el sistema, cómo se comprobará su funcionamiento y de qué manera se organizará internamente. A partir del análisis de los requerimientos fue posible establecer las principales necesidades de los estudiantes y funcionarios y convertirlas en funcionalidades que pueden ser verificadas mediante casos de prueba.

La aplicación de pruebas permitió identificar comportamientos correctos y situaciones en las que el sistema debe mejorar. La mayoría de las funcionalidades planteadas cuentan con escenarios positivos y negativos, lo que permite tener una visión más completa del comportamiento esperado. Sin embargo, también se identificaron aspectos pendientes, como la falta de pruebas específicas para algunos requerimientos y la necesidad de normalizar la relación entre determinados requerimientos y sus respectivos casos de prueba. Esto demuestra la importancia de mantener una correcta trazabilidad durante el desarrollo.

En cuanto al diseño, la utilización de MVC y la separación de responsabilidades permiten organizar mejor el proyecto y evitar que toda la lógica termine concentrada en un único lugar. En la propuesta realizada, el frontend, backend y base de datos se encuentran separados, mientras que dentro del backend se distribuyen responsabilidades entre controladores, servicios y modelos. Esto facilita el mantenimiento y permite que diferentes integrantes puedan trabajar sobre áreas específicas sin afectar directamente las demás. También se pudo comprobar la importancia de la modularidad, el bajo acoplamiento y la alta cohesión. La arquitectura propuesta permite que componentes como notificaciones, autenticación, solicitudes, persistencia y reportes tengan responsabilidades independientes. Por ejemplo, si en el futuro se cambia el mecanismo de notificaciones, no sería necesario modificar todo el sistema, ya que los demás componentes interactúan mediante interfaces y no dependen directamente de la implementación interna.

Finalmente, el análisis de una posible segunda fase con un módulo de PQRS y notificaciones por correo permitió comprobar que una arquitectura bien estructurada facilita la evolución del software. La propuesta permite reutilizar componentes existentes y agregar nuevas funcionalidades sin tener que reconstruir todo el sistema, aplicando principios como Open/Closed y Dependency Inversion.

En conclusión, este trabajo permitió entender de una manera más práctica la relación entre los requerimientos, las pruebas, el diseño y la arquitectura de software. Más que construir una solución que simplemente funcione, el objetivo debe ser desarrollar un sistema organizado, mantenible y preparado para crecer, ya que las decisiones tomadas durante el diseño tienen un impacto directo en la facilidad con la que el software podrá modificarse y mantenerse en el futuro.